

Apostila do Aluno

Mundo de Cubos

UC12 · Curso Técnico em Informática
Aula 24 · C# com um mundo 3D no navegador

8 métodos para escrever · **1 monstro** para desenhar

Aluno: _____

Turma: _____ Data: ____/____/____

Sumário

TODO 1	Mover
TODO 2	Pular
TODO 3	Minerar
TODO 4	Cavar
TODO 5	Colocar
TODO 6	CriarBlocos
TODO 7	CriarInimigos
TODO 8	PerseguirJogador

O que é isto

Um mundo 3D que abre no navegador — ilha, morros, praia, lago e árvores — e que **não faz absolutamente nada** até você escrever o código.

O navegador só desenha e escuta o teclado. Quem **decide** tudo é um programa em C# que roda na sua máquina, e dentro dele os **oito métodos que você vai escrever**. Enquanto um método está vazio, a tecla dele não funciona e a tela avisa qual desafio destrava aquilo.

Não é prova. Não vale nota. É a última aula do módulo, e a ideia é que você termine a noite com um jogo que é seu — com o seu nome no título, os seus blocos na paleta e um monstro que você desenhou correndo atrás de você.

Como usar esta apostila

Cada método tem um capítulo, e todo capítulo tem as mesmas sete partes:

Parte	Para quê
O que este método faz	Uma frase. Se você entendeu, já dá para tentar
Por que ele existe assim	A decisão por trás. É aqui que se aprende
O código	O método inteiro, pronto
Linha a linha	O porquê de cada linha
O erro mais comum	O que a maioria erra, e como o teste pega
Como saber que funcionou	O que olhar na tela
Para pensar	Uma pergunta sem resposta escrita

O código está em imagem, e é de propósito. Você não consegue copiar e colar — nem da tela, nem do PDF. O caminho do código até o seu programa passa pelos seus dedos, e é exatamente aí que ele entra na sua cabeça.

A regra que vale a noite inteira

escreve o método -> salva -> PARA o programa -> F5 de novo

Parar é obrigatório, e é o passo que todo mundo esquece.

O `Desafios.cs` é *compilado*. Enquanto o servidor estiver rodando, ele continua usando o código de antes — o que você acabou de escrever nem existe para ele. O botão *Conferir de novo* recalcula, mas com o mesmo código, e por isso ele avisa isso em letras miúdas.

Se você mudou o código, salvou, apertou *Conferir de novo* e nada mudou, não é o programa que está errado: é que ele ainda não foi recompilado.

As três regras do `Desafios.cs`

- 1. Nenhum método seu sorteia nada.** O dado, o acaso, a semente — tudo chega por parâmetro. Um método que sorteia por dentro não pode ser testado, porque a resposta muda a cada execução.
- 2. Nenhum método seu desenha nada.** Sem `MessageBox`, sem `Console.WriteLine`. Quem desenha é o navegador. Todo método **recebe** valores e **devolve** uma resposta.
- 3. Nenhum método seu chama outro do mesmo arquivo.** Se o `Cavar` chamasse o `Minerar`, um erro no `Minerar` pintaria o `Cavar` de vermelho — e o diagnóstico na tela estaria mentindo para você.

O mundo, e por que você nunca vê uma matriz

O mundo é uma caixa de **24 x 20 x 24** cubinhos.

```
Y (para cima, ate 19)
|
|   Z (para o fundo, ate 23)
|  /
| /
| /
+----- X (para os lados, ate 23)
```

- **Y = 0** é o bedrock, o fundo do mundo. Não quebra, nem com a melhor picareta.
- **Você ocupa DUAS casas:** os pés em `Y` e a cabeça em `Y + 1`. É por isso que quase toda checagem de passagem neste projeto é dupla.
- **A água e a folha deixam passar.** Você entra no lago e afunda até o fundo, e passa por baixo da copa das árvores sem enroscar.

Guardar cubos em três dimensões precisa de uma **matriz**, e matriz é assunto que a turma não viu. Então ela existe — mas fica **privada dentro da classe** `Mundo`, e você conversa com o mundo por métodos:

Você chama	Ele responde
<code>mundo.Dentro(x, y, z)</code>	essa casa existe na caixa?
<code>mundo.Vazio(x, y, z)</code>	dá para passar por ela?
<code>mundo.Bloco(x, y, z)</code>	o nome do bloco, ou <code>""</code> se for ar
<code>mundo.Dureza("pedra")</code>	quanto de picareta esse tipo pede
<code>mundo.Por(x, y, z, tipo)</code>	põe um bloco
<code>mundo.Tirar(x, y, z)</code>	tira o bloco

Isso tem nome, e você já viu: é **encapsulamento**. O mesmo motivo pelo qual a senha do `Usuario`, no projeto passado, não era um campo público. O `Mundo` guarda a matriz e decide o que você pode fazer com ela.

As duas fábricas, e por que não há construtor

Quando você precisar de um lugar novo ou de um bloco novo, use:

<code>Pos.Nova(x, y, z)</code>	faz um lugar
<code>Bloco.Novo(nome, cor, dureza)</code>	faz um tipo de bloco
<code>Inimigo.Novo(desenho, x, z)</code>	faz um monstro

Você nunca escreve `new Bloco("neon", "#39FF14", 1)`. Isso é *construtor*, e construtor é assunto que a turma não viu — está inclusive na lista do que ficou de fora, lá na Aula 13.

`Bloco.Novo` é um **método estático**, que é exatamente o que você escreve desde a primeira aula: recebe valores, devolve uma coisa pronta.

Antes de programar: os dois arquivos de brincar

Estes dois não têm nada para aprender. São para você deixar o mundo com a sua cara — e é bom fazer isso **antes** do primeiro método, porque é uma vitória rápida e o resto da noite fica mais divertido.

MinhaConfig.cs

Você muda	O que acontece
<code>Jogador</code>	O seu nome vira o título do jogo, lá em cima
<code>Semente</code>	A ilha inteira muda: morros, praia, lago, árvores
<code>CorDaRoupa</code> , <code>CorDaCalca</code>	A roupa do seu boneco
<code>CorDaPele</code> , <code>CorDoCabelo</code>	O rosto e o cabelo dele
<code>CorDoCeu</code>	Ponha <code>#101018</code> e o seu mundo vira noite

As cores são hexadecimais da web: um `#` e seis dígitos de `0` a `9` ou de `A` a `F`. Os dois primeiros são vermelho, os dois do meio verde, os dois últimos azul.

Nada aí pode quebrar o jogo. Se você escrever uma cor torta, o motor usa a de fábrica e segue em frente.

MeuMonstro.cs

Aqui você **desenha** o seu bicho. Ele é feito de cubinhos, em fatias empilhadas:

```
"a...a",      <- os pes, primeira fatia
".....",
"---",        <- acabou este andar
".aaaa.",     <- a barriga
"aaaaaa",
```

Cada letra (**a**, **b**, **c**, **d**) é um cubinho de uma das suas quatro cores, e o ponto é vazio. O **---** diz "acabou este andar, começa o de cima". A primeira fatia são os pés; a última é o alto da cabeça.

O monstro entra no jogo no **desafio 7**, com o nome **"meumonstro"**.

Um desenho torto nunca derruba o jogo. Linha de tamanho errado é completada, letra desconhecida vira vazio, e se você apagar tudo vem um cubinho no lugar.

Como o jogo abre

1. Abra o **MundoDeCubos.csproj** no Visual Studio.
2. **F5**.
3. O Visual Studio sobe o servidor **e abre o navegador sozinho** em **http://localhost:5199**.

Vai aparecer uma **janela preta de console**. Não feche — é o servidor. Para parar, feche ela ou aperte o quadrado vermelho no Visual Studio.

Não precisa instalar nada. Este projeto não usa nenhum pacote do NuGet: o ASP.NET Core já vem dentro do .NET 8. Ele funciona com a internet fora do ar.

A armadilha avisada

Quando você rodar pela primeira vez, o painel vai dizer coisas assim:

4 de 7 testes passando

Você ainda não escreveu nada. Como isso passou em quatro?

Porque um método vazio devolve o **valor padrão do tipo**: um **int** volta **0**, um **bool** volta **false**, uma **string** volta **""**. E alguns testes esperam justamente esses valores.

Só vale 100%. O selo verde é o único sinal que conta. Quatro de sete não é "quase lá" — é "não comecei".

O desafio 5 é o campeão nisso: **return false;** já acerta cinco dos sete testes, porque cinco deles são recusas.

`int nx = onde.X + dx;` e `int nz = onde.Z + dz;` — o destino, calculado uma vez só. Some primeiro, decida depois: tentar decidir tudo antes da soma vira uma cadeia de `if` que ninguém lê.

`if (!mundo.Dentro(nx, onde.Y, nz))` — a casa existe? O `!` lê-se "não": se *NÃO estiver dentro*. Repare que o `Y` não muda aqui, porque andar não sobe nem desce por si só.

`return onde;` — devolve **a mesma posição que chegou**. Não é "não faz nada": o método precisa devolver um lugar, e o lugar certo é onde ele já estava.

`mundo.Vazio(nx, onde.Y, nz) && mundo.Vazio(nx, onde.Y + 1, nz)` — as duas casas do corpo. O `&&` só é verdadeiro quando as duas são. Pés livres e cabeça livre.

`return Pos.Nova(nx, onde.Y, nz);` — um lugar novo, com o mesmo `Y`.

`mundo.Vazio(nx, onde.Y + 1, nz) && mundo.Vazio(nx, onde.Y + 2, nz)` — o degrau. Se chegamos até aqui, é porque a casa dos pés está ocupada. Então olhamos uma casa acima: se **ela** e a de cima dela estiverem livres, o corpo inteiro cabe lá em cima.

`return Pos.Nova(nx, onde.Y + 1, nz);` — sobe um cubo. É a única linha do método que muda o `Y`.

`return onde;` — o último. Passou por tudo e nada valeu: é parede.

O erro mais comum

Olhar só os pés. O `if` vira:

```
if (mundo.Vazio(nx, onde.Y, nz))
{
    return Pos.Nova(nx, onde.Y, nz);
}
```

Funciona em quase tudo, e o teste "*parede de dois cubos barra a passagem*" pega: o boneco entra de cabeça na pedra.

Erro nº 2: esquecer o degrau. Passa em 6 dos 7 testes e o jogo fica horrível de jogar — o boneco trava a cada cubo de morro.

Erro nº 3: subir sempre que tiver bloco na frente, sem checar se cabe. Aí o boneco escala paredes de qualquer altura, como se fosse aranha.

Como saber que funcionou

O selo do desafio 1 fica verde e as teclas **W A S D** passam a andar. Vá até um morro: o boneco sobe os degraus de um cubo sozinho, e para nas paredes altas.

Para pensar

O `Mover` não faz o boneco cair — quem faz isso é o motor, depois. Por que separar as duas coisas? O que aconteceria com os testes se cair fosse trabalho deste método?

O erro mais comum

Usar `y + 1` no lugar de `y + 2`. É o erro clássico, e ele é geométrico: com `y + 1` você está perguntando se a casa da *cabeça* está livre — e ela está, é a cabeça dele. O boneco atravessa o teto.

O teste *"teto colado na cabeça: não sobe nada"* pega isso.

Erro nº 2, e este trava o programa: esquecer de somar 1 no contador. A condição nunca fica falsa e o laço roda para sempre. O servidor espera 2 segundos, desiste, e escreve na tela:

"O seu código passou de 2 segundos sem terminar. Quase sempre é um laço cuja condição nunca fica falsa: confira se alguma coisa muda DENTRO do corpo do while."

Os **outros desafios continuam funcionando** — um erro num não derruba os demais.

Como saber que funcionou

A barra de espaço passa a pular. Construa um bloco acima da sua cabeça (depois do desafio 5) e tente pular embaixo dele: o boneco não sobe.

Para pensar

Com força zero, o `while` não entra nenhuma vez e o método devolve o `Y` que recebeu. Alguns alunos põem um `if (força == 0) return onde.Y;` antes do laço. Não está errado — mas por que é desnecessário? E qual seria a diferença se o laço fosse um `do-while`?

Linha a linha

`if (!mundo.Dentro(x, y, z)) { return ""; } —` a mira pode apontar para fora do mundo. Texto vazio quer dizer "não caiu nada".

`string tipo = mundo.Bloco(x, y, z); —` guarda o nome numa variável. Vamos usar esse nome três vezes; pedir três vezes ao mundo seria desperdício e deixaria o código mais longo.

`if (tipo == "") { return ""; } —` mirou no ar.

`if (tipo == "bedrock") { return ""; } —` o chão do mundo não quebra.

`if (picareta < mundo.Dureza(tipo)) { return ""; } —` ferramenta fraca demais. Repare: se esta linha viesse antes das outras, `mundo.Dureza("")` responderia 1 e o método "quebraria o ar".

`mundo.Tirar(x, y, z); —` o bloco sai do mundo. **Esta linha é a que muda o jogo;** tudo depois dela é só decidir o que dizer.

`switch (tipo) —` o valor a comparar.

`case "pedra": return "cascalho"; —` cada `case` termina com `return`, então não precisa de `break`: o `return` já sai do método inteiro.

`case "folha": return ""; —` a folha **some**. O bloco sai do mundo do mesmo jeito, mas nada cai na sua mão. É a única regra do jogo em que quebrar não rende nada.

`default: return tipo; —` e este é o mais importante do método. É o `default` que faz os blocos que **você** inventar no desafio 6 caírem com o próprio nome, sem você escrever um `case` para cada um.

O erro mais comum

Perguntar a dureza antes de saber se existe bloco. O método passa a "quebrar o ar" e o teste do bedrock falha.

Erro nº 2: devolver o nome sem chamar `mundo.Tirar`. O corretor pega na hora, porque ele confere o que caiu **e** o que ficou no lugar:

```
esperado: [cascalho / ar]
obtido:   [cascalho / pedra]
```

Essa linha sozinha explica o erro: o nome está certo, mas a pedra continua lá. Sempre leia o "obtido".

Como saber que funcionou

O clique esquerdo passa a quebrar blocos. Mine uma pedra e leia o aviso: vai dizer **cascalho**, não pedra. Mine uma folha: o bloco some e o aviso não aparece.

Para pensar

Por que o `default` devolve o próprio tipo em vez de texto vazio? O que aconteceria com os seus blocos do desafio 6 se ele devolvesse `""`?

`if (mundo.Bloco(x, alvo, z) == "bedrock") { return quantos; }` — chegou ao fundo do mundo. Para aqui.

`if (!mundo.Vazio(x, alvo, z))` — a casa tem alguma coisa? Se estiver vazia, o laço continua para a próxima, sem contar.

`mundo.Tirar(x, alvo, z);` e `quantos = quantos + 1;` — tira e conta. Sempre juntos: contar sem tirar, ou tirar sem contar, dá o número errado.

`return quantos;` — o total, quando o laço termina normalmente.

O erro mais comum

Declarar o contador dentro do `for`:

```
for (int i = 0; i < profundidade; i++)
{
    int quantos = 0;
    ...
}
```

O compilador **não reclama**. A variável simplesmente nasce de novo a cada volta e volta zerada, e o método devolve 0 ou 1 para sempre. Se o seu número está teimando em 1, é isto.

Erro nº 2: contar casa vazia. O cenário `buraco` dos testes existe só para isso: profundidade 5, mas só 3 blocos tirados porque duas casas já eram ar.

Como saber que funcionou

A tecla **C** abre um poço embaixo da mira. Cave embaixo dos próprios pés e você cai dentro do buraco — a gravidade é do motor e funciona sozinha.

O resultado do teste mostra a coluna inteira, lida de baixo para cima, uma letra por bloco: `bppppp.....` é bedrock, cinco de pedra e seis de ar.

Para pensar

O `return` dentro do `for` sai do método inteiro, não só do laço. Como você escreveria este método se quisesse **parar o laço mas continuar o método** depois dele? Que palavra o C# tem para isso, e por que aqui ela não é necessária?

TODO 5 - Colocar

Revisa: `if` ċ Destrava: o clique direito

O que este método faz

Põe um bloco na casa mirada e devolve `true` — ou recusa e devolve `false`.

Por que ele existe assim

Este é o primeiro método do projeto que devolve `true` ou `false` em vez de um valor. Quem chama precisa saber se deu certo para poder avisar na tela — e é por isso que o tipo de retorno é `bool` e não `void`.

São quatro recusas. As três primeiras são óbvias: fora do mundo, casa ocupada, tipo vazio. Nenhuma delas precisa de `else`: são `if` soltos, um embaixo do outro, cada um com o seu `return false`. Quando a resposta é "desisto", o `return` é mais limpo que o `else`.

A quarta recusa é a interessante, e ela é sobre **você**: se desse para colocar bloco em cima de si mesmo, dava para se empregar dentro da pedra e o jogo acabava ali. Como o boneco ocupa duas casas, a recusa também é dupla.

E repare no que **não** é recusado: colocar bloco **acima da cabeça** vale. É assim que se constrói para cima.

O código

```
public static bool Colocar(Mundo mundo, int x, int y, int z,
                           string tipo, Pos jogador)
{
    if (!mundo.Dentro(x, y, z)) { return false; }
    if (!mundo.Vazio(x, y, z)) { return false; }
    if (tipo == "") { return false; }

    if (x == jogador.X && z == jogador.Z)
    {
        if (y == jogador.Y || y == jogador.Y + 1) { return false; }
    }

    mundo.Por(x, y, z, tipo);
    return true;
}
```

Linha a linha

`if (!mundo.Dentro(x, y, z)) { return false; }` — a mira pode apontar para fora da caixa.

`if (!mundo.Vazio(x, y, z)) { return false; }` — casa ocupada não recebe outro bloco. Sem esta linha você pintaria a pedra de verde em vez de construir.

`if (tipo == "") { return false; }` — sem esta, blocos sem nome entram no mundo e você não faz ideia do porquê.

`if (x == jogador.X && z == jogador.Z)` — a coluna do jogador. Ainda não recusa nada: só delimita **onde** olhar.

`if (y == jogador.Y || y == jogador.Y + 1) { return false; }` — as duas casas do corpo. O `||` lê-se "ou": os pés **ou** a cabeça.

`mondo.Por(x, y, z, tipo);` e `return true;` — passou pelas quatro recusas: põe o bloco e avisa que deu certo.

O erro mais comum

Esquecer a casa da cabeça. O `if` fica só com `y == jogador.Y`, e aí dá para se emparedar por cima. O teste *"em cima da própria cabeça, não"* pega.

Erro nº 2: recusar tudo o que está perto do jogador, inclusive acima da cabeça. Aí não dá para construir para cima, que é metade da graça do jogo.

Como saber que funcionou

O clique direito passa a colocar o bloco escolhido na paleta. E agora o desafio 5 virou a sua **defesa**: quando os monstros chegarem, no desafio 8, você pode se cercar de blocos.

Só que uma parede de **um** cubo não basta — o monstro sobe degrau igual a você. Precisa de **dois**.

Para pensar

Este método recebe a posição do jogador só para não deixar você se emparedar. Um método que recebe uma informação só para dizer "não" a ela — isso é bom desenho, ou é sinal de que a regra deveria morar em outro lugar?

De 1 a 10 blocos	Menos que um, o teste recusa; mais que dez também
Nome sem espaço	<code>pedra magica</code> não vale; <code>pedramagica</code> vale
Nome que ainda não existe	<code>pedra</code> , <code>terra</code> , <code>grama</code> , <code>areia</code> , <code>agua</code> , <code>tronco</code> , <code>folha</code> , <code>cascalho</code> e <code>bedrock</code> já são de fábrica
Nome não repetido	Dois blocos com o mesmo nome, não
Cor <code>#RRGGBB</code>	Um <code>#</code> e seis dígitos. O CSS aceita três; este teste não
Dureza de 1 a 5	Fora disso, recusa

O teste não compara com uma lista fixa — seus blocos são seus. Ele confere o **formato** e diz exatamente qual está torto:

```
(cor invalida em lava: #FF00)
(nome repetido: gelo)
(esse nome ja e de fabrica: pedra)
(dureza fora de 1 a 5 em ouro: 9)
```

O erro mais comum

Tentar `new Bloco("neon", "#39FF14", 1)`. Não compila — a classe `Bloco` não tem construtor com parâmetros, de propósito. Use `Bloco.Novo`.

Erro nº 2: cor com três dígitos (`#F0A`) ou sem o `#`. O formato é `#RRGGBB`, seis dígitos, sempre.

Como saber que funcionou

Os seus blocos aparecem **na frente** na paleta, com borda verde, e as teclas `1`, `2`, `3`... caem neles primeiro. Coloque um no mundo e veja a cor que você escolheu.

Para pensar

Comece com **um** bloco só, rode, veja aparecer, e só então acrescente os outros. Por que essa ordem é melhor do que escrever os dez de uma vez e rodar no fim?

E: se você criar um bloco de dureza 5, você mesmo não consegue quebrá-lo. Isso é um defeito do jogo ou é uma regra?

TODO 7 - CriarInimigos

Revisa: List<T> · **Destrava:** os monstros aparecem

O que este método faz

Solta no mundo os monstros que você escolheu — inclusive o que você desenhou.

Por que ele existe assim

É **gêmeo do desafio 6**, e isso é de propósito: quem fez os blocos faz os monstros em dois minutos. O valor deste desafio não é a dificuldade — é a diversão. Você já sabe o molde; só muda a fábrica.

O **Inimigo.Novo** recebe **três** coisas: o desenho, o **x** e o **z**. **Você não escreve o Y**, e isso é uma decisão do motor: ele solta o monstro do céu e deixa cair até o chão. Se você mandasse o **Y**, um monstro criado em cima de um morro nasceria dentro da pedra.

O motor vai além: se o lugar que você pediu estiver dentro de uma árvore ou apertado demais, ele **procura em volta** uma coluna onde um bicho de dois cubos caiba de pé.

O código

```
public static List<Inimigo> CriarInimigos()
{
    List<Inimigo> meus = new List<Inimigo>();

    meus.Add(Inimigo.Novo("gosma", 5, 5));
    meus.Add(Inimigo.Novo("aranha", 19, 5));
    meus.Add(Inimigo.Novo("meumonstrro", 19, 19));

    return meus;
}
```

Os cinco desenhos

Nome	O bicho
"gosma"	uma bolha verde de olhos pretos
"fantasma"	branco, com franjas onduladas embaixo
"aranha"	preta, de pernas e olhos vermelhos
"robo"	cinza, com antena
"meumonstro"	o seu , aquele do <code>MeuMonstro.cs</code>

Linha a linha

```
List<Inimigo> meus = new List<Inimigo>(); — a lista vazia, igual à do desafio 6.
```

`meus.Add(Inimigo.Novo("gosma", 5, 5));` — desenho, `x` e `z`. Os dois números vão de 0 a 23, como qualquer casa do mundo.

`return meus;` — de 1 a 6 monstros.

O erro mais comum

Escrever um desenho que não existe — `"zumbi"`, `"dragao"`. O teste diz exatamente qual:

`(desenho desconhecido: zumbi)`. São cinco nomes, e só cinco.

Erro nº 2: tentar passar o `Y`. A fábrica só tem três parâmetros.

Como saber que funcionou

Os monstros aparecem no mundo. Eles ainda **não se mexem** — isso é o desafio 8. Aproveite para dar uma volta e olhar cada um de perto.

Para pensar

Ponha os monstros **longe do meio**, que é onde você nasce. Os cantos, como `(5, 5)` e `(19, 19)`, são bons. Por que a primeira impressão do desafio 8 fica ruim se os quatro monstros nascerem em cima de você?

```
public static Pos PerseguirJogador(Mundo mundo, Pos inimigo, Pos jogador)
{
    int dx = 0;
    int dz = 0;

    if (jogador.X > inimigo.X) { dx = 1; }
    else if (jogador.X < inimigo.X) { dx = -1; }
    else if (jogador.Z > inimigo.Z) { dz = 1; }
    else if (jogador.Z < inimigo.Z) { dz = -1; }

    int nx = inimigo.X + dx;
    int nz = inimigo.Z + dz;

    if (mundo.Vazio(nx, inimigo.Y, nz) && mundo.Vazio(nx, inimigo.Y + 1, nz))
    {
        return Pos.Nova(nx, inimigo.Y, nz);
    }

    if (mundo.Vazio(nx, inimigo.Y + 1, nz) && mundo.Vazio(nx, inimigo.Y + 2, nz))
    {
        return Pos.Nova(nx, inimigo.Y + 1, nz);
    }

    return inimigo;
}
```

Linha a linha

`int dx = 0;` e `int dz = 0;` — o passo em cada eixo, começando parado. Se nenhum `if` pegar, os dois ficam zero e o monstro não sai do lugar — que é exatamente o certo quando ele já está em cima de você.

`if (jogador.X > inimigo.X) { dx = 1; }` — você está a leste: ele vai para o leste.

`else if (jogador.X < inimigo.X) { dx = -1; }` — a oeste.

`else if (jogador.Z > inimigo.Z) { dz = 1; }` — o `else if` é o que faz o Z só ser olhado **quando o X já está resolvido**. Se estes quatro fossem `if` soltos, o monstro decidiria `dx` e `dz` na mesma rodada e andaria na diagonal.

`int nx = inimigo.X + dx;` e `int nz = inimigo.Z + dz;` — o destino.

Os dois `if` seguintes — copiados do seu `Mover`: anda reto, ou sobe o degrau.

`return inimigo;` — parede de dois cubos. O bicho para.

O erro mais comum

Usar `if` soltos em vez de `else if`. O monstro anda na diagonal, e o teste "*você na diagonal: ele acerta o X PRIMEIRO*" pega:

```
esperado: [6/9/5]
obtido:   [6/9/6]
```

Erro nº 2: esquecer o degrau. Passa em 7 dos 8 testes, e no jogo os monstros ficam presos no primeiro morro. Você acha que a perseguição não funciona — e ela funciona; o bicho é que está preso.

Como saber que funcionou

Os monstros começam a vir na sua direção a cada passo seu. Quando um encostar, o aviso aparece e o contador "**pego Nx**" sobe no topo da tela.

Encostar não machuca: é a última aula, não um jogo de sobrevivência. Mas tente não deixar.

Para pensar

A ordem X-antes-de-Z é uma **decisão de projeto**, não uma verdade. Se você inverter — Z primeiro, X depois — o monstro caça de um jeito diferente, e não está errado. Só que os testes esperam esta.

Como você faria um monstro que anda **na diagonal**? E por que isso deixaria o jogo mais difícil do que parece?

Terminou os oito. E agora?

O jogo é seu. Algumas coisas para tentar antes de a aula acabar:

Troque a semente	Cada número dá uma ilha diferente. Ache uma bonita e mostre
Ponha o mundo à noite	<code>CorDoCeu = "#101018"</code> no <code>MinhaConfig.cs</code>
Faça uma torre	Empilhe os seus blocos e olhe de longe com a roda do mouse
Se tranque numa caixa	Dois cubos de altura. Veja os monstros baterem na parede
Cave até o bedrock	O fundo do mundo não quebra. Confira
Desenhe outro monstro	O <code>MeuMonstro.cs</code> aceita mudança a qualquer hora
Crie um bloco de dureza 5	E descubra que você mesmo não consegue quebrá-lo

O que este projeto revisou

Construção	Onde
<code>if</code> solto, com <code>return</code>	1, 3, 4, 5
Cadeia <code>if / else if</code>	8
<code>while</code> com contador e guarda	2
<code>for</code> com contagem conhecida	4
<code>switch</code> com <code>case</code> e <code>default</code>	3
<code>List<T></code> : criar, <code>Add</code> , devolver	6 e 7
Método com parâmetro e retorno	todos
<code>&&</code> e <code>`</code>	<code>`</code> 1, 2, 5
Encapsulamento (conversar com o <code>Mundo</code>)	todos

Ficaram de fora, de propósito: `foreach`, `TryParse` com `out`, `try/finally` e banco de dados. Todos foram exercitados nos projetos anteriores.

Se der problema

O que acontece	O que é
"A página não abre"	O servidor não subiu. Olhe a janela preta do console
"Diz que não tem WebGL"	Navegador antigo. Use o Chrome ou o Edge
"Travou por 2 segundos"	<code>while</code> sem passo, no desafio 2
"Mudei o código e não muda nada"	Não parou o servidor. O <code>Desafios.cs</code> é compilado
"A porta 5199 já está em uso"	Sobrou um servidor de antes. Feche a janela preta
"O programa fechou sozinho"	Um método chamando a si mesmo. Confira o <code>MeuMonstro.cs</code> e o desafio 6

Não usa

Herança, polimorfismo, interfaces, **construtor**, vetores, matrizes, ordenação escrita à mão, LINQ, lambdas, `async`, `throw`. Nada disso foi ministrado no módulo, e nada disso é necessário aqui. Se a sua solução precisou de algo dessa lista, existe um caminho mais simples — e vale perguntar qual.

E **você não precisa saber HTML, CSS nem JavaScript**. A tela inteira vem pronta. O que você escreve é C#.